Authors:
Gabriela Błaut,
Michał Brzeziński,
Aleksandra Gołek

KittyProtocol |Phase 1

1. Purpose and scope of the protocol

★ Purpose (What is the protocol used for?):

The goal of KittyProtocol is to enable secure, ephemeral, and instant exchange of short text messages in a client-server architecture, while:

- no message history stored on the server,
- no cryptographic keys stored on the server,
- full end-to-end encryption performed exclusively on the client side,
- resistance to eavesdropping, tampering, and man-in-the-middle attacks.

The protocol requires that the two clients establish a shared secret (Pre-Shared Secret, PSS) outside the system (out-of-band), e.g., through a face-to-face meeting, a phone call, a QR code, or another secure channel.

★ Problems Solved:

The protocol is a lightweight alternative to resource-intensive, traditional messengers focusing on maximum privacy and speed. It solves two key network problems

★ **High latency:** By leveraging the QUIC transport layer, KittyProtocol significantly minimizes connection setup time.

★ **No native support for mobility:**
The protocol addresses the issue of session disconnections during frequent network changes. It ensures session continuity and maintains communication without requiring the user to log in again.

And security-related issues:

- **Man-in-the-middle (MitM) attack:** The ability to impersonate a user during communication
- **Confidentiality:** No device other than the sender and recipient is able to read the message
- **Integrity:** Thanks to the use of the QUIC protocol, which utilizes TLS 1.3, it prevents the integrity of transmitted frames from being compromised.

Communication model:

Communication takes place in a **client-server (C/S)** model using full end-to-end (E2EE) encryption. The central server (Hub) acts as a message router and identity manager. This architecture completely resolves the issue of node visibility in NAT tables. Since the server only forwards asymmetrically encrypted payloads, it is impossible to read the content of conversations.

2. Technical Specifications

Transport:

The protocol uses the QUIC transport layer. We chose QUIC because it combines the speed of UDP with the reliability of TCP. A key factor was the 0-RTT connection establishment mechanism, which significantly reduces connection setup time. . While 0-RTT is vulnerable to replay attacks, and QUIC/TLS 1.3 does not guarantee that data sent in 0-RTT is replay-resistant. However, this is addressed by a unique msg_id. Additionally, by using a single data stream per connection, we ensure the order of incoming messages. This approach also provides native TLS 1.3 encryption and connection stability when switching networks (e.g., from Wi-Fi to LTE) thanks to the use of Connection ID.

Message encoding:

We chose the JSON text format as our encoding format. It combines human readability with high parsing efficiency. This choice facilitates debugging (the frame structure is easily visible, for example, in Wireshark) and eliminates low-level Byte-order issues (endianness) characteristic of binary protocols. JSON's flexibility allows for easy future expansion of the protocol with new fields without compromising backward compatibility.

Reliability assumptions:

Reliability-related tasks have been clearly divided between the transport and application layers.

- **What the transport layer (QUIC) provides:**

  It guarantees the integrity of the delivered data, maintains the correct order of packets within individual streams, and automatically retransmits lost packets

★ **What does the application protocol (Kitty Protocol) provide:**

- **Logical acknowledgments (Application-level ACK):**
  The MEOW_OK message is sent by the recipient only after the application has fully and correctly validated the JSON structure. This means that the message was not only delivered (transport layer ACK from QUIC), but also understood and accepted.
- **Rate Limiting:**
  We limit the number of messages to 10 per second per user (and a maximum of 100 messages per minute from a single IP address) to prevent spam/DoS attacks. Testing: e.g., sending 1,000 requests to a node/backend server.
- **Delivery ACK:**
  Delivery ACK: In C/S communication, the recipient sends a message receipt confirmation (Delivered), indicating that the data was successfully received at the transport layer. This does not guarantee its processing by the application (application layer handling).
- **Application timeouts:**
  We are implementing a strict 20-second time limit for completing the authentication process. Failure to respond results in an ERR_03 (Timeout) error and the session being closed unilaterally.

3. Message Structure (Formats)

★ Message types:

The following main types of control and data exchange messages have been defined (values for the type field):

★ HELLO – **Session initialization.**
The client initiates an application session with the Hub by sending a HELLO message containing only the message identifier msg_id. No cryptographic keys are transmitted at this stage - this is used solely to initiate the session and prepare for the authentication process.

★ AUTH – user authentication in the Hub.

★ GET_STATUS - **Client request to the Hub for the status of the intended recipient.**
This frame is used to check whether the specified user is currently online and can receive messages.

★ STATUS_RES - **The Hub's response to the GET_KEY request.** It contains information whether the recipient is online.

★ DATA – transmission of the actual payload (text/ASCII images) between users.

- ★ MEOW_O – universal application-level acknowledgment (Application-level ACK).
- ★ ERROR – reporting validation, authentication, or timeout errors.
- ★ PING – Keep-alive messages sent to the server to maintain the session.
- ★ BYE – correct and intentional termination of the session.

★ Message fields (required and optional):

Each frame is a valid JSON object.

Required fields (in every message):

- Type (String) – specifies the message type (e.g., "DATA").
- msg_id (timestamp) – a unique message identifier within a session. It prevents duplicates at the application level and is used for mapping acknowledgments (the recipient includes the same msg_id when sending back (MEOW_OK). Instead of the traditional msg_id as an incremented integer, the message identifier within the session is directly **the timestamp** (e.g.,290326213840). The QUIC transport layer handles the retransmission of lost packets.

Specific/optional fields (depending on type):

- payload (String) – the actual message content, required for the DATA field
- mac (String) – HMAC of the message
- user, pass (String) – login credentials, required only for AUTH
- target (String) – recipient identifier, required in the DATA and GET_STATUS frames.
- code Desc (String) – error code and description; appear only in ERROR messages

★ Format (example of a general frame):

All messages are based on a single consistent base structure. Example of a complete frame with data:

```
{
      "type": "DATA",
      "msg_id": 1709048120311,
      "target": "bob",
      "payload": "BASE64(ciphertext)",
      "mac": "BASE64(HMAC_SHA256(payload || msg_id || target))"
}
```

Architectural note:
The HMAC is calculated using a separate, end-to-end key shared between clients (unknown to the server). Therefore, the HMAC can serve as an additional layer of E2E integrity / MitM detection over the transport layer.

★ What must remain in the frame?

- "type": "DATA" - The QUIC protocol operates on byte streams and does not impose any semantics on the transmitted data. The application layer must therefore unambiguously specify the message type to enable its correct interpretation (e.g., text message, PING, BYE).

- "msg_id": timestamp - Although QUIC ensures transmission reliability and data ordering within a single stream, the message identifier is essential at the application logic level. It enables:

  - unambiguously link a message to its corresponding acknowledgment (MEOW_OK),
  - handling potential retransmissions at the application level,
  - correctly reflecting the message status in the user interface (e.g., the “Delivered” label).

  Additionally, the “Timestamp” itself specifies the time the message was created and plays a significant role in application logic, in particular:

  - it enables the detection and rejection of outdated or replayed messages,
  - supports the optional ordering of messages regardless of transport mechanisms,
  - facilitates debugging and event analysis in a distributed system.

- "payload": "Meow! ASCII art." - Contains the actual application data transmitted between peers. QUIC does not interfere with the structure or interpretation of this field.

★ Validation (what we consider a format error):

The application rigorously checks every incoming frame. The following are considered format errors (resulting in immediate message rejection and the return of an ERROR message with code ERR_02):

- Parsing error: The received string is not a valid JSON document (e.g., missing brackets, quotation marks).
- Missing base fields: The JSON object is missing the `type` or `msg_id` key.
- Unknown type: The type key contains a value not in the allowed list.
- Variable type mismatch: For example, the msg_id field was sent as a string instead of an integer.

4. State model and communication flow

★ Session description (Communication flow):

The lifecycle of each session in the KittyProtocol is divided into 4 distinct phases:

Phase 1 – Handshake (HELLO)

Client ➜ Hub:

```
{ "type": "HELLO", "msg_id": 1709048120300 }
```

Hub ➜ Client:

```
{ "type": "MEOW_OK", "msg_id": 1709048120301,
  "status": "Ready for auth" }
```

Phase 2 – Login (AUTH)

Client ➜ Hub:

```
{ "type": "AUTH", "msg_id": 1709048120310, "user":
"alice", "pass": "secret" }
```

Hub ➜ Client:

```
{ "type": "MEOW_OK", "msg_id": 1709048120311,
  "status": "Logged in" }
```

The server stores **only the username and password (hash)** .

Phase 3 – Checking the recipient's status

Client ➜ Hub:

```
{ "type": "GET_STATUS", "msg_id": 1709048120320, "target": "bob" }
```

Hub ➜ Client:

```
{ "type": "STATUS_RES", "msg_id": 1709048120321,
  "target": "bob", "status": "online" }
```

Phase 4 – Data exchange (E2EE)

Establishing a shared secret (OOB – out of band) Alice and Bob establish the secret outside the system:

K_AB = 32-byte shared secret

From this, they derive:

K_enc = KDF(K_AB, "encryption")

K_mac = KDF(K_AB, "authentication")

Sending a message

Alice encrypts the plaintext:

cipher = **AEAD_Encrypt** (K_enc, nonce=msg_id plaintext) mac

= **HMAC_SHA256** (K_mac cipher || msg_id || target)

Alice ➜ Hub:

```
{ "type": "DATA", "msg_id": 1709048120330, "target": "bob",
  "payload": "BASE64(cipher)","mac": "BASE64(mac)"}
```

Hub ➜ Bob:

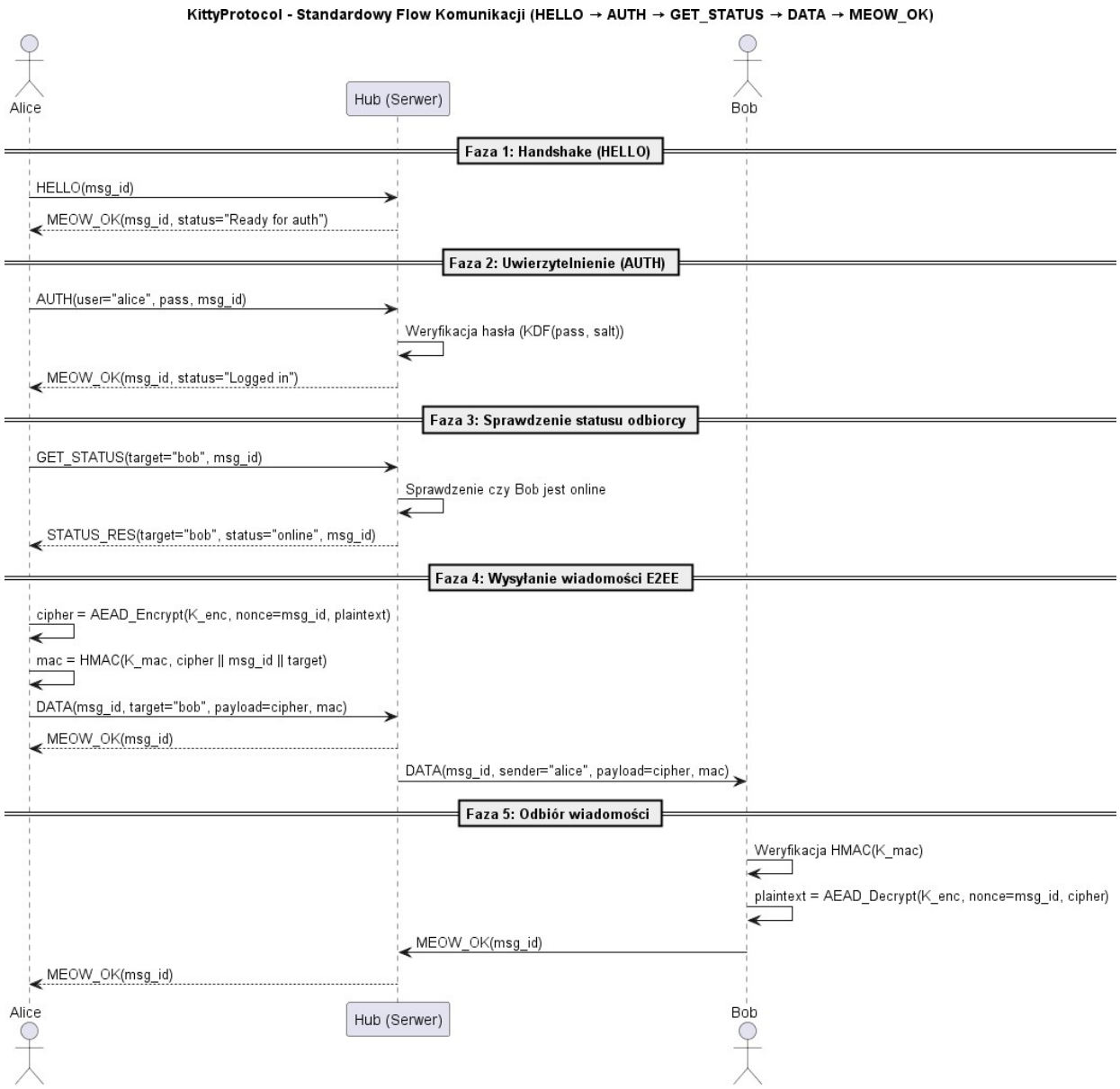
```
{ "type": "DATA", "msg_id": 1709048120330,"sender": "alice",
  "payload": "BASE64(cipher)", "mac": "BASE64(mac)"}
```

Bob:

- verifies the HMAC ,
- decrypts the message ,
- sends back MEOW_OK.

★ Sample sequence diagram:

The diagram illustrates the correct message flow from connection establishment to data exchange.



★ **Keep-alive and reliability mechanisms:**

- Keep-alive: To prevent network devices (NAT/Firewall) from closing inactive QUIC streams, a rule was introduced requiring the client to send a PING message every 30 seconds.
- Timeouts: Authentication timeout: A strict 20-second limit has been imposed for the successful transmission of the AUTH message following the sending of the HELLO message. Exceeding this time limit results in an ERR_03 error and the termination of the connection
- Session timeout: If the receiver does not receive any data (DATA or PING messages) from the sender within 60 seconds, the session is unilaterally closed (the host is considered offline).
- Retransmission / Retry: At Layer 4 (QUIC), the retransmission of lost packets is performed fully automatically and transparently to the application.

At the application level: if the sender sends a DATA message and does not receive a MEOW_OK (Application-level ACK) from the server within 5 seconds, the application **does not** blindly **retry** the transmission (since QUIC is natively responsible for retransmitting lost packets). Instead, the message receives a "Not Delivered/Timeout" status in the user interface, suggesting a connection loss on the part of the other host.

5. Security

★ Confidentiality and Transport Integrity:

The QUIC transport layer (with TLS 1.3 enforced) ensures full confidentiality and integrity of all transmitted data—both the JSON structure and the encrypted payload. QUIC uses AEAD (Authenticated Encryption with Associated Data) mechanisms, which guarantee:

- detection of any in-flight data tampering,
- protection against eavesdropping,
- protection against classic Man-in-the-Middle attacks.

The application does not need to implement additional HMACs or checksums—QUIC provides this natively.

★**Data confidentiality** (E2EE): KittyProtocol uses end-to-end encryption at the application level.

The mechanism works as follows:

1. Clients agree on a shared secret (e.g., a private key).
2. The sender encrypts the message **using the shared secret** .
3. The hub merely forwards the encrypted DATA frames—it has no access to the clients' private keys.

As a result:

- The hub cannot read the message content,
- a server compromise does not reveal the conversation history, and the entire content is visible only to the sender and recipient.
- Not even the Hub can impersonate any of the devices.

★ Authentication

The login process in KittyProtocol involves verifying the user's identity using a username and password.

1. The client sends an DATA frame to the Hub containing:
    - user – the username,
    - pass – the password in plaintext within an encrypted QUIC/TLS 1.3 channel.
2. QUIC/TLS 1.3 ensures full confidentiality and integrity of the transport, so the password is never sent in plaintext outside the secure tunnel.
3. On the server side, the password is processed as follows:
    - the server retrieves the password_hash and salt from the database,
    - calculates KDF(pass, salt) (e.g., Argon2 / bcrypt),
    - compares the result with the stored hash.
4. If the values match, the server considers the user authenticated and returns MEOW_OK.

Example database table structure:

```
CREATE TABLE users (

    id SERIAL PRIMARY KEY,

    username VARCHAR(50) UNIQUE NOT NULL,

    password_hash VARCHAR(255) NOT NULL,
-- Hashed using bcrypt/rsa, for example!

    is_online BOOLEAN DEFAULT FALSE,
-- Online status flag (optional, as the Hub can store this in RAM)

);
```

★ Protection against replay attacks:

1. Transport layer security (TLS 1.3)

TLS 1.3 protects against packet replay within a QUIC session.

2. Protection at the application layer (msg_id + timestamp)

Each message contains: **msg_id** — a unique identifier (timestamp). The client maintains a list of recently processed msg_ids in RAM, so if a duplicate appears, the frame is silently discarded.

This protects against:

- ★ 0-RTT replay attacks,
- ★ injection of old frames,
- ★ application-side replay attacks.

★ Threat model:

KittyProtocol is designed to operate in untrusted network environments (e.g., public Wi-Fi networks).

- **Eavesdropping, Man-in-the-Middle (MitM), and Tampering:**

  Fully mitigated natively by QUIC/TLS 1.3 and E2EE

- **Resource Exhaustion (DoS Attacks / Spam):**

  Threats minimized through:

  - **Rate limiting** on the Hub side:
    - max 10 messages/s per user,
    - max 100 messages/min from a single IP.
  - strict validation of JSON frames,
  - immediate session termination upon protocol violation.

- **Compromise of the central server (Hub):**

  Even if the Hub is compromised, the message content remains confidential (E2EE).

  An attacker can only:

  - interfere with routing,
  - block users,
  - manipulate signaling.

  They cannot

  - Decrypt messages,
  - read Call history,
  - eavesdrop on communications.

6. Error and Connection Failure Handling

Errors in KittyProtocol are communicated asynchronously using ERROR frames, allowing the application to respond immediately.

★ Error codes

Based on the document, the following codes can be distinguished:

Code	Meaning	When it occurs	Recommended action
ERR_01	Protocol Violation	E.g., Sending DATA before AUTH	End session, start with HELLO
ERR_02	Format Error	Invalid JSON Missing type/msg_id incorrect types	Correct the frame; end the session after Multiple errors
ERR_03	Authorization Timeout	No AUTH within **20 seconds** of HELLO	Display timeout, retry login
ERR_04	Authentication Failed	Invalid login credentials	Request correct credentials
ERR_05	Session error	Undefined session error	Force re-authentication and attemptto reconnect
ERR_06	Replay Detected	Reuse msg_id	Discard the frame, optionally close the session
ERR_07	Rate Limit Exceeded	>10 messages/s or >100/min from IP	Apply backoff, wait
ERR_08	Delivery Failed – Recipient Offline	The recipient is not online	Notify sender, no queuing

ERR_09	Session Timeout (Idle)	No activity for 60 seconds	Re-establish connection or perform handshake
ERR_10	Resource Exhaustion	Hub overload	Retry with random jitter
ERR_11	Internal Server Error	Hub error (e.g., DB offline)	Retry after a short time
ERR_12	Version Mismatch	Unsupported protocol version	Update client
ERR_13	Payload Too Large	Payload > 2048 bytes	Trim the text and resend
ERR_14	Not Authorized	No permissions for this action	Abort operation
ERR_15	Unknown Target	GET_KEY for a Non-existent user	Abort the send attempt

★ Behavior after syntax/protocol errors:

- Each JSON frame undergoes full structural validation.
- In the event of a formatting error, the recipient immediately returns an ERROR (ERR_02) .
- Repeated errors or protocol violations result in unilateral session termination .

★ Connection timeouts:

- **Authentication timeout:**

  The AUTH process must complete within **20 seconds** of HELLO. If the limit is exceeded:

  - The Hub returns ERR_03 ,
  - the session is closed,
  - resources are released.

- **Idle timeout:**
  The hub monitors client activity.

  If no DATA or PING is received within 60 seconds:

  - the session is considered terminated,
  - the Hub closes the QUIC connection,
  - and the client must perform a new handshake.

Timeouts help mitigate the risk of replay and DoS attacks by automatically closing inactive connections. Monitoring inactivity on the server side ensures data privacy the server has no visibility into the plaintext content of Exchanged DATA messages.

★ Connection loss during a session

QUIC provides:

- resilience to brief interruptions,
- migration between networks (Connection ID),
- automatic retransmissions.

In the event of a permanent loss of connection:

- the client performs a new handshake,
- it can resume the session,
- the application can use the last msg_id to mark messages as undelivered.

★ Duplicates and incomplete messages

- **Protection against duplicates:**
  - unique msg_id
  - client-side caching of recent identifiers (e.g., in RAM)

  Duplicates are silently rejected.

★ Incomplete messages

- QUIC guarantees stream integrity.
- A malformed JSON structure will result in an ERR_02 error.

★ Limits and Abuse Protection

- **Rate Limiting and IP Protection:**
  - Max **1 message/sec** per user,
  - Max **100 messages/min** from a single IP.

- **Message size:**
  - maximum JSON frame size: **4 KB,**
  - maximum payload size after encryption: **2048 bytes**

- **Truncation (at the client-side application level):**
  If a user attempts to send a message larger than 2 KB, the client application immediately truncates the excess characters at the interface/logic level (before encryption) and sends a DATA frame that fits within the limit over the network. This prevents errors and rejections on the Hub side.

★ Error Frame Format

In KittyProtocol, errors are communicated via **ERROR** frames containing the following fields:

- type: "ERROR",
- msg_id,
- code – error code/short description of the error type,
- desc – error description

7. Scenarios (Frame Examples)

Scenario 1: Successful session and message exchange (E2EE via the Hub)

The standard flow from establishing a connection with the Hub was presented earlier in section 4: **State model and communication flow**

Scenario 2: Format validation error (ERR_02)

The client sends invalid JSON (missing type, incorrect msg_id type).

Client → Hub

```
{
  "msg_id": 1709048120330,
  "payload": "Encrypted text..."
}
```

Hub → Client

```
{
  "type": "ERROR",
  "msg_id": "1709048120335",
  "code": "ERR_02",
  "desc": "Required 'type' field missing or type mismatch"
}
```

Scenario 3: Authentication process timeout (ERR_03)

Client → Hub

```
{
  "type": "HELLO",
  "msg_id": "1709048120500",
}
```

Hub → Client

```
{
  "type":      "MEOW_OK”,
  "msg_id": "1709048120505"
}
```

(Client has not sent an AUTH for 20 seconds)

Hub → Client

```
{
  "type": "ERROR",
  "msg_id": "1709048120500",
  "code": "ERR_03",
  "desc": "Authorization timeout reached"
}
```

Scenario 4: Failed login (ERR_04)

Client → Hub


```
{
  "type": "AUTH",
  "msg_id": "1709048120312",
  "user": "hacker",
  "pass": "123"
}
```


Hub → Client


```
{
  "type": "ERROR",
  "msg_id": "1709048120318",
  "code": "ERR_04",
  "desc": "Authentication failed"
}
```


Scenario 5: Recipient offline (ERR_15)

Client → Hub


```
{
  "type": "DATA",
  "msg_id": "1709048120400",
  "target":"bob",
  "payload": "R3g4... [ciphertext]"
}
```

Hub → Client

```
{
  "type": "ERROR",
  "msg_id": "1709048120406-A6",
  "code": "ERR_15",
  "desc": "Receiver offline"
}
```

Scenario 6: Message longer than 2 KB

The client application detects that the **2048-byte** limit has been exceeded and truncates the text locally.

Client → Hub

```
{
  "type": "DATA",
  "msg_id": "1709048120500",
  "target":"bob",
  "payload": "R3g4... [ciphertext of the truncated message]"
}
```

Hub → Client

```
{
  "type":     "MEOW_OK”,
  "msg_id": "1709048120508"
}
```